

FreelanceControl

Manual completo del sistema

Trabajo Final de Grado — Ingeniería en Sistemas de Información

Versión al 31/05/2026

FreelanceControl es un sistema de gestión financiera diseñado específicamente para monotributistas argentinos. Integra clasificación automática de gastos con ML local, proyección de ingresos con Prophet, monitoreo fiscal del Monotributo, auditoría automática de registros, importación de extractos bancarios y generación de reportes PDF mensuales.

Campo	Valor
Stack	Python 3.11 · FastAPI · PostgreSQL 15 · React
ML / IA	scikit-learn (SVM/NB) · Prophet · Groq llama-3.3-70b
Infraestructura	Docker Compose · Alembic (5 migraciones)
Tests	97 tests · pytest · SQLite in-memory
Credenciales demo	demo@freelancecontrol.com / demo1234
API base URL	http://localhost:8000
Frontend	http://localhost:3000
Documentación API	http://localhost:8000/docs

Índice de contenidos

1	Descripción del proyecto	3
2	Arquitectura del sistema	4
3	Instalación y puesta en marcha	5
4	Módulos funcionales	7
4.1	Autenticación y usuarios	7
4.2	Gestión de ingresos	8
4.3	Gestión de gastos con clasificación ML	8
4.4	Gestión de facturas	10
4.5	Importación de extractos bancarios	11
4.6	Auditoría automatizada	13
4.7	Proyección de ingresos (Prophet)	15
4.8	Monitoreo fiscal del Monotributo	16
4.9	Resumen financiero con IA (Groq)	17
4.10	Reportes PDF mensuales	18
5	Referencia de la API REST	19
6	Manual de usuario paso a paso	22
7	Guía de troubleshooting	25
8	Preguntas frecuentes (FAQ)	26

1. Descripción del proyecto

FreelanceControl nació para resolver un problema concreto: los trabajadores independientes argentinos que tributan bajo el régimen de Monotributo carecen de herramientas accesibles para gestionar su actividad financiera. Las soluciones comerciales existentes son costosas, diseñadas para empresas más grandes o no contemplan las particularidades del régimen fiscal argentino.

El sistema aborda tres objetivos concretos: (1) automatizar la categorización de gastos sin depender de servicios externos, preservando la privacidad del usuario; (2) anticipar el riesgo de recategorización fiscal combinando datos históricos con proyecciones; (3) reducir la carga operativa integrando la importación de extractos bancarios y la generación de reportes exportables.

1.1 Principios de diseño

- **Soberanía de datos:** la descripción de los gastos NUNCA sale del sistema. La clasificación corre 100% local con un modelo SVM entrenado sobre 600 ejemplos.
- **Fallback local siempre:** si Groq no está disponible, el resumen y las recomendaciones se generan con plantillas determinísticas. El sistema no falla.
- **Inmutabilidad de facturas pagadas:** una vez marcada como PAGADA, una factura no puede editarse ni eliminarse (HTTP 409). Protege el historial contable.
- **usuario_id siempre del JWT:** ningún endpoint acepta el usuario en el body. Previene acceso cruzado entre usuarios.
- **Anti-enumeración en login:** el mismo error para email inexistente y password incorrecto. No revela qué cuentas existen.

1.2 Cobertura funcional

HU	Historia de usuario	Estado
HU-01	Registro de usuario	✓
HU-02	Inicio de sesión con JWT	✓
HU-03	Registro de ingresos con filtros	✓
HU-04	Registro de gastos con clasificación ML	✓
HU-05	Corrección manual de categorías (reentrenamiento)	✓
HU-06	Gestión de facturas con máquina de estados	✓
HU-07	Importación masiva CSV/Excel desde el banco	✓
HU-08	Auditoría automatizada (4 detectores)	✓
HU-09	Proyección de ingresos a 6 meses (Prophet)	✓
HU-10	Monitoreo fiscal Monotributo con semáforo	✓
HU-11	Resumen financiero en lenguaje natural	✓

HU-12	Recomendaciones personalizadas sobre datos agregados	✓
HU-13	Reporte mensual descargable en PDF	✓

2. Arquitectura del sistema

El sistema sigue una arquitectura de tres capas contenedorizadas con Docker Compose. La capa de presentación (React) se comunica exclusivamente con la API REST (FastAPI), que a su vez gestiona la persistencia en PostgreSQL. No hay comunicación directa entre el frontend y la base de datos.

Capa	Tecnología	Puerto	Responsabilidad
Presentación	React (Node 18 Alpine)	3000	Interfaz de usuario con 13 pantallas
API REST	FastAPI + Uvicorn (Python 3.11)	8000	Lógica de negocio, ML, auth JWT
Persistencia	PostgreSQL 15	5432	Datos, modelos ML, historial

2.1 Stack tecnológico completo

Componente	Tecnología	Versión
Framework web	FastAPI	0.110.0
Servidor ASGI	Uvicorn	0.29.0
ORM	SQLAlchemy	2.0.29
Validación	Pydantic	2.6.4
Auth JWT	python-jose + passlib/bcrypt	3.3.0 / 1.7.4
ML clasificador	scikit-learn (SVM/NB + TF-IDF)	1.4.2
Proyecciones	Prophet + pandas	1.1.5 / 2.2.1
IA externa	Groq llama-3.3-70b-versatile	0.13.0
Reportes PDF	ReportLab	4.1.0
Importación	pandas + openpyxl	2.2.1 / 3.1.2
Migraciones	Alembic	1.13.1
Tests	pytest + httpx + pytest-cov	8.1.1
Contenedores	Docker + Docker Compose 3.8	—

2.2 Estructura de la base de datos

El esquema cuenta con 8 tablas principales, gestionadas con 5 migraciones Alembic:

Tabla	Descripción	Relaciones clave
usuarios	Perfil, email único, hash bcrypt, categoría Monotributo	Padre de todo
ingresos	Cobros: descripción, monto, categoría, fecha	→ usuarios
gastos	Gastos: ídem + flag es_duplicado	→ usuarios

facturas	Estado (PENDIENTE/PAGADA/VENCIDA), fechas, clientes	→ usuarios
proyecciones	Monto proyectado + intervalo de confianza por mes	→ usuarios
alertas_auditoria	Tipo alerta, descripción, resuelta (bool)	→ usuarios
cache_clasificacion	Correcciones manuales: descripción normalizada	→ categorías
modelos_clasificador	Modelo ML serializado (base o personalizado por usuario)	→ usuarios (nullable)
categorias_monotributo	Límites y cuotas A–K vigentes desde 2026-02-01 Independiente	

3. Instalación y puesta en marcha

3.1 Requisitos previos

- Docker Desktop 4.x o superior (incluye Docker Compose v2)
- Git (para clonar el repositorio)
- 4 GB de RAM disponible para los contenedores
- Puertos 3000, 8000 y 5432 libres

3.2 Clonar y configurar

```
git clone <URL_DEL_REPOSITORIO> cd proyecto-tfg cp .env.example .env
```

■ Editar `.env` antes de levantar. Como mínimo: `DATABASE_URL` (ya pre-configurada para Docker), `SECRET_KEY` (cambiar en producción) y opcionalmente `GROQ_API_KEY` para habilitar la IA.

3.3 Variables de entorno (.env)

Variable	Valor de ejemplo	Descripción
POSTGRES_USER	marcos	Usuario de PostgreSQL
POSTGRES_PASSWORD	marcos123	Contraseña de PostgreSQL
POSTGRES_DB	tfg_freelancers	Nombre de la base de datos
DATABASE_URL	postgresql://marcos:marcos123@db:5432/	URL completa de conexión
SECRET_KEY	clave-aleatoria-de-64-caracteres	Clave para firmar JWT
ALGORITHM	HS256	Algoritmo de firma del token
ACCESS_TOKEN_EXPIRE_MINUTES	7680	Vigencia del token (7 días)
GROQ_API_KEY	gsk_...	API key de Groq (opcional, para IA)
GROQ_MODEL	llama-3.3-70b-versatile	Modelo de Groq a usar

3.4 Levantar el sistema

```
# Construir imágenes y levantar los 3 servicios docker compose up -d # Verificar que los servicios están corriendo docker compose ps # Las migraciones Alembic se ejecutan automáticamente al iniciar la API # Verificar que la API respondió: curl http://localhost:8000/health # Respuesta esperada: {"status":"ok","servicio":"TFG Freelancers API"}
```

3.5 Poblar datos iniciales (una sola vez)

```
# 1. Cargar categorías de Monotributo (A-K, valores 2026) docker compose exec api python seed_categorias_monotributo.py # 2. Entrenar el modelo base de clasificación de gastos docker compose exec api python seed_modelo_base.py # 3. Reiniciar la API para que tome el modelo en memoria docker compose restart api # 4. (Opcional) Cargar datos de demostración
```

```
docker compose exec api python seed_demo.py # Login demo: demo@freelancecontrol.com / demo1234
```

■ `seed_demo.py` es **IDEMPOTENTE**: se puede correr múltiples veces sin romper nada. Borra el usuario demo anterior y recrea todos los datos desde cero.

3.6 Acceder al sistema

Servicio	URL	Descripción
Frontend	http://localhost:3000	Interfaz React (13 pantallas)
API REST	http://localhost:8000	FastAPI — endpoint raíz
Swagger UI	http://localhost:8000/docs	Documentación interactiva de la API
ReDoc	http://localhost:8000/redoc	Documentación alternativa
Health check	http://localhost:8000/health	Estado del servicio

4. Módulos funcionales

4.1 Autenticación y usuarios

El sistema usa autenticación basada en JWT (JSON Web Token) con vigencia de 7 días. Las contraseñas se almacenan con hash bcrypt y nunca en texto plano. El token se incluye en cada solicitud como header

Authorization: Bearer <token>.

- **Registro (POST /auth/register)**: valida formato de email, verifica unicidad, hashea la contraseña y devuelve 201 con el perfil creado.
- **Login (POST /auth/login)**: usa OAuth2PasswordRequestForm. Devuelve el MISMO error si el email no existe o la contraseña es incorrecta (anti-enumeración).
- **Perfil de usuario**: incluye nombre, email, categoría Monotributo y actividad (servicios/comercio).

4.2 Gestión de ingresos

Los ingresos representan los cobros del freelancer. Tienen descripción, monto, categoría y fecha. El usuario_id se toma siempre del JWT, nunca del body.

- CRUD completo: crear, listar, obtener por ID, actualizar y eliminar.
- Listado ordenado por fecha descendente con filtros opcionales por categoría.
- Paginación: parámetros ?limite y ?offset.

4.3 Gestión de gastos con clasificación automática (ML)

El módulo de gastos es el corazón del sistema. Combina CRUD estándar con un pipeline completo de clasificación automática usando procesamiento de lenguaje natural 100% local.

Flujo de clasificación

- El usuario ingresa solo la **descripción** del gasto.
- El sistema invoca el endpoint **POST /gastos/clasificar** que ejecuta el pipeline ML localmente.
- Se devuelve la categoría sugerida junto con un **valor de confianza** (0-1).
- Si la confianza está **por debajo de 0.30**, se devuelve 'Otros' y se marca el gasto como **sujeto a revisión manual**.
- El usuario puede confirmar o cambiar la categoría antes de guardar.

Orden de resolución del clasificador

Paso	Descripción	Resultado
1	Busca la descripción normalizada en las correcciones previas	Devuelve categoría + confianza 1.0 (fuente: correccion_usuario)
2	Invoca el modelo ML local (SVM sobre TF-IDF vectorizado)	Devuelve categoría + confianza
3	Si confianza < 0.30	Devuelve 'Otros' + requiere_revision=true

Modelos de clasificación

- **Modelo base:** SVM lineal + TF-IDF sobre 600 ejemplos etiquetados en 12 categorías. Precisión media por cross-validation: ~76%.
- **Modelo personalizado:** cuando el usuario acumula 20+ gastos propios, se entrena un modelo que combina el dataset base con sus datos reales.
- **Reentrenamiento:** se dispara en background cada vez que el usuario crea o edita un gasto. No bloquea la respuesta.

Categoría	Ejemplos de gastos
Software	Licencias, IDEs, herramientas SaaS profesionales
Hardware	Equipos, periféricos, componentes
Infraestructura	Hosting, servidores, dominios, cloud
Marketing	Publicidad digital, diseño, redes sociales
Servicios	Honorarios profesionales (contador, abogado)
Capacitación	Cursos, libros técnicos, certificaciones
Suscripciones	Netflix, Spotify, Adobe CC, herramientas mensual
Transporte	Uber, nafta, estacionamiento, vuelos
Alimentación	Almuerzos de trabajo, delivery, cafés
Impuestos	Ingresos brutos, ganancias, AFIP
Monotributo	Pago de la cuota mensual
Otros	Sin categoría clara o confianza baja

4.4 Gestión de facturas

Las facturas representan los servicios emitidos a clientes. Tienen una máquina de estados que garantiza la integridad del historial contable.

Estado	Descripción	Transiciones posibles
PENDIENTE	Estado inicial al crear	→ PAGADA (con fecha_pago) → VENCIDA (por auditoría)
PAGADA	Cobro registrado con fecha	Sin más transiciones (inmutable)
VENCIDA	Superó fecha de vencimiento sin cobrar	→ PAGADA (si cobran tarde)

■ Una factura PAGADA NO puede editarse ni eliminarse (HTTP 409). Esta restricción protege el historial contable ante errores accidentales.

4.5 Importación de extractos bancarios (CSV / Excel)

El módulo de importación permite cargar movimientos bancarios de múltiples bancos argentinos sin configuración manual. La detección de columnas y separador es completamente local, sin enviar datos a servicios externos.

Banco	Formato	Separador	Fecha	Montos
Galicia	CSV	; (punto y coma)	dd/mm/yyyy	Débito / Crédito separados
Santander	CSV	, (coma)	yyyy-mm-dd	Importe único con signo
Brubank	CSV	, (coma)	yyyy-mm-dd	Monto con signo +/-
BBVA / Macro / Nación	CSV/XLSX	Auto-detectado	Varios	Varios
Mercado Pago	CSV/XLSX	Auto-detectado	Varios	Varios

Flujo de importación

- **POST /importar/preview:** sube el archivo, detecta columnas automáticamente, marca posibles duplicados, clasifica las primeras 20 filas y devuelve la vista previa.
- **POST /importar/confirmar:** recibe los movimientos aprobados por el usuario y los persiste en una transacción atómica. Si falla alguno, se revierte todo.

Restricciones de seguridad

- Extensiones permitidas: **.csv** y **.xlsx**. Cualquier otra → HTTP 400.
- Tamaño máximo: **10 MB**. Archivos más grandes → HTTP 413.
- La detección de duplicados compara por (fecha, monto, descripción normalizada). Un gasto que aparece N veces en el CSV y ya existe M veces en BD → solo marca los primeros M como duplicados.

4.6 Auditoría automatizada (Módulo M3)

La auditoría se ejecuta bajo demanda (POST /alertas/ejecutar-auditoria) y corre cuatro detectores en secuencia. Borra las alertas no resueltas anteriores para evitar duplicados, preservando las ya resueltas como historial.

Detector	Criterio	Resultado
1 — Gastos duplicados	Mismo monto, categoría Y descripción normalizada	Alerta de GASTO_DUPLICADO con monto, categoría y descripción
2 — Anomalías estadísticas	$z\text{-score} > 2.0$ respecto de la media de la categoría	Alerta de ANOMALIA_ESTADISTICA con monto, media y desviación estándar
3 — Discrepancias de facturación	Facturas en estado PENDIENTE cuya fecha_vencimiento > fecha_pago	Alerta de DISCREPANCIA_FACTURACION por cada factura
4 — Monotributo impago	No existe gasto con categoria='Monotributo' en el mes en curso	Alerta de MONOTRIBUTO_IMPAGO con cuota esperada

■ El detector de anomalías estadísticas requiere al menos 5 gastos en la misma categoría para que la desviación estándar sea estadísticamente confiable. Con menos datos no actúa.

4.7 Proyección de ingresos (Prophet)

El módulo de proyecciones genera estimaciones de ingresos para los próximos 6 meses con intervalos de confianza (límite inferior y superior). Usa Facebook Prophet cuando hay suficientes datos históricos, o media móvil simple como fallback.

Condición	Estrategia	Resultado
≥ 10 ingresos históricos	Prophet (modelo de series temporales)	Proyección con tendencia y estacionalidad
< 10 ingresos históricos	Media móvil simple (cold start)	Proyección conservadora basada en promedio

- Cada generación **reemplaza** las proyecciones anteriores del usuario.
- Las proyecciones se almacenan con **monto_lower** y **monto_upper** (intervalo de confianza al 80%).
- El estado de Monotributo usa las proyecciones para calcular el porcentaje proyectado al cierre del año fiscal.

4.8 Monitoreo fiscal del Monotributo

El módulo de Monotributo calcula dinámicamente el riesgo de recategorización combinando la facturación real del año con la proyección Prophet hasta el 31 de diciembre.

Semáforo	Condición	Significado
VERDE	Porcentaje proyectado $< 70\%$	Sin riesgo inmediato
AMARILLO	Porcentaje proyectado $70\% - 90\%$	Zona de precaución, monitorear
ROJO	Porcentaje proyectado $> 90\%$	Riesgo alto de recategorización

Categorías A–K (2026)

Cat.	Límite anual	Cuota mensual (servicios)
A	\$ 10.277.988	\$ 42.387
B	\$ 15.068.988	\$ 48.251
C	\$ 21.010.988	\$ 56.502
D	\$ 27.540.988	\$ 72.414
E	\$ 34.650.988	\$ 102.548
F	\$ 45.280.988	\$ 129.045
G	\$ 56.510.988	\$ 174.378
H	\$ 79.130.988	\$ 447.347
I	\$ 94.500.988	\$ 606.019
J	\$ 101.430.988	\$ 805.938
K	\$ 108.357.084	\$ 1.080.000

4.9 Resumen financiero y recomendaciones (IA — Groq)

El sistema usa Groq/LLaMA 3.3 EXCLUSIVAMENTE para generar texto narrativo sobre **datos numéricos agregados**. Las descripciones individuales de gastos o facturas NUNCA se transmiten a servicios externos.

Endpoint	Qué envía a Groq	Fallback si no hay API key
GET /resumen/financiero	Totales de ingresos/gastos por categoría, facturas pendientes	Reporte generado con plantillas determinísticas
GET /recomendaciones/	Conteo de alertas por tipo, variaciones % de gastos, total proyectado	Recomendaciones estándar basadas en

4.10 Reportes PDF mensuales

El endpoint GET /reportes/pdf genera un archivo PDF descargable con el resumen financiero del período indicado. Se usa ReportLab de forma programática (sin HTML).

Sección del PDF	Contenido
Encabezado	Nombre del usuario, período, fecha y hora de generación
Resumen ejecutivo	Ingresos totales, gastos totales, balance, variación vs mes anterior
Estado fiscal	Categoría Monotributo, límite anual, cuota, estado del pago del mes
Distribución de gastos	Tabla por categoría con monto, % del total y cantidad
Facturación del período	Emitidas/pagadas/pendientes/vencidas con montos
Auditoría	Resumen por tipo de alerta + detalle con descripción y monto
Pie de página	Numeración, disclaimer: no reemplaza asesoramiento contable

■ Todos los montos en el PDF usan formato argentino: separador de miles con punto y decimales con coma. Ejemplo: \$ 1.234.567,89

5. Referencia de la API REST

Todos los endpoints (excepto `/auth/register` y `/auth/login`) requieren el header **Authorization: Bearer <token>**. El token se obtiene en el login. La documentación interactiva completa está disponible en <http://localhost:8000/docs>.

Autenticación

Método	Ruta	Entrada	Respuesta
POST	<code>/auth/register</code>	UsuarioCreate	201 UsuarioResponse
POST	<code>/auth/login</code>	OAuth2PasswordRequestForm	200 Token

Ingresos

Método	Ruta	Entrada	Respuesta
POST	<code>/ingresos/</code>	IngresoCreate	201 IngresoResponse
GET	<code>/ingresos/</code>	?categoria, limite, offset	list[IngresoResponse]
GET	<code>/ingresos/{id}</code>	—	IngresoResponse
PUT	<code>/ingresos/{id}</code>	IngresoCreate	IngresoResponse
DELETE	<code>/ingresos/{id}</code>	—	204

Gastos

Método	Ruta	Entrada	Respuesta
POST	<code>/gastos/clasificar</code>	{descripcion}	{categoria_sugerida, confianza, requiere_revision}
POST	<code>/gastos/</code>	GastoCreate	201 GastoResponse
GET	<code>/gastos/</code>	?categoria, solo_duplicados, limite, offset	list[GastoResponse]
GET	<code>/gastos/{id}</code>	—	GastoResponse
PUT	<code>/gastos/{id}</code>	GastoCreate	GastoResponse
DELETE	<code>/gastos/{id}</code>	—	204

Facturas

Método	Ruta	Entrada	Respuesta
POST	<code>/facturas/</code>	FacturaCreate	201 FacturaResponse
GET	<code>/facturas/</code>	?estado, cliente_nombre, limite, offset	list[FacturaResponse]
GET	<code>/facturas/{id}</code>	—	FacturaResponse
PUT	<code>/facturas/{id}</code>	FacturaUpdate	FacturaResponse (409 si PAGADA)
PATCH	<code>/facturas/{id}/estado</code>	FacturaEstadoUpdate	FacturaResponse
DELETE	<code>/facturas/{id}</code>	—	204 (409 si PAGADA)

Alertas / Auditoría

Método	Ruta	Entrada	Respuesta
POST	/alertas/ejecutar-auditoria	—	{mensaje, detalle}
GET	/alertas/	?tipo, solo_pendientes, limite, offset	list[AlertaResponse]
GET	/alertas/{id}	—	AlertaResponse
PATCH	/alertas/{id}/resolver	AlertaResolverUpdate	AlertaResponse

Proyecciones

Método	Ruta	Entrada	Respuesta
POST	/proyecciones/generar	ProyeccionGenerarRequest	list[ProyeccionResponse]
GET	/proyecciones/	?limite, offset	list[ProyeccionResponse]
GET	/proyecciones/{id}	—	ProyeccionResponse

Monotributo

Método	Ruta	Entrada	Respuesta
GET	/monotributo/estado	—	{categoria, limite, porcentaje_usado, estado, ...}
GET	/monotributo/pago	—	{pagado, mes, monto_esperado, ...}
PATCH	/monotributo/categoria	UsuarioUpdateMonotributo	UsuarioResponse
GET	/monotributo/facturacion-12-meses	—	{facturacion_12_meses, desde, hasta}

IA / Resumen

Método	Ruta	Entrada	Respuesta
GET	/resumen/financiero	?mes, anio	{resumen: str, generado_con_ia: bool}
GET	/recomendaciones/	—	{recomendaciones: list[str], generado_con_ia: bool}

Importación

Método	Ruta	Entrada	Respuesta
POST	/importar/preview	UploadFile	{total_filas, preview, mapeo_detectado, resumen}
POST	/importar/confirmar	ConfirmarRequest	{importados, ingresos_creados, gastos_creados, ...}

Clasificador ML

Método	Ruta	Entrada	Respuesta
GET	/ml/estado	—	{algoritmo, precision, n_ejemplos, tipo_modelo}
POST	/ml/reentrenar	—	Resultado del reentrenamiento
POST	/ml/corregir	{descripcion, categoria_correcta}	{mensaje, nuevo_estado}

Reportes

Método	Ruta	Entrada	Respuesta
GET	/reportes/pdf	?mes, año	application/pdf (attachment)

6. Manual de usuario paso a paso

6.1 Primer uso — onboarding

Seguir estos pasos al registrarse por primera vez:

Paso	Acción	Resultado esperado
1	Abrir <code>http://localhost:3000</code> → 'Registrarse'	Formulario de registro
2	Completar nombre, email y contraseña (mín. 8 chars)	Cuenta creada
3	Iniciar sesión con las credenciales	Dashboard principal
4	Ir a Configuración → Monotributo → asignar categoría	Monotributo fiscal activo
5	Ir a Gastos → Clasificador → probar con una descripción	Verificar que el ML clasifica
6	Registrar primer ingreso con descripción, monto y fecha	Aparece en el listado

6.2 Flujo diario recomendado

- **Al recibir un pago:** registrar ingreso con el nombre del cliente, el monto y la fecha del cobro.
- **Al pagar algo:** ir a Gastos → escribir la descripción → confirmar o ajustar la categoría sugerida por el ML → guardar.
- **Al emitir una factura:** registrarla en Facturas con fecha de emisión y vencimiento. Cuando cobres, cambiar el estado a PAGADA.
- **Una vez al mes:** ejecutar la auditoría para detectar anomalías y duplicados. Generar el reporte PDF para enviar al contador.

6.3 Importar extracto del banco

- Descargar el extracto desde el homebanking (CSV o Excel).
- Ir a Importar → subir el archivo.
- Revisar la vista previa: verificar que las columnas fueron detectadas correctamente y que los tipos (ingreso/gasto) son coherentes.
- Desmarcar los movimientos que sean duplicados o que no quieras importar.
- Confirmar la importación. El sistema persiste todo en una sola transacción.

■ Si el sistema no detecta las columnas, verificá que el archivo tenga encabezados en la primera fila. Los nombres de columna más comunes (fecha, concepto, débito, crédito, importe, monto) son reconocidos automáticamente.

6.4 Corregir una clasificación del ML

- Opción A: al crear el gasto, simplemente cambiar la categoría sugerida antes de guardar.
- Opción B: editar un gasto ya guardado y cambiar su categoría.
- Opción C: ir a la pantalla Clasificador → escribir la descripción y la categoría correcta → Corregir. Esto guarda la corrección permanentemente.

En todos los casos, la corrección se persiste en la base de datos y se usa en el próximo reentrenamiento. A partir de entonces, la misma descripción (con variantes de mayúsculas, tildes o espacios) siempre devolverá esa categoría con confianza 1.0, sin invocar el ML.

6.5 Interpretar el semáforo del Monotributo

El semáforo se basa en el **porcentaje proyectado** (no el ya facturado). Combina todo lo facturado en el año + las proyecciones Prophet hasta el 31 de diciembre.

- **Verde:** la proyección anual está por debajo del 70% del límite. Sin acción necesaria.
- **Amarillo:** la proyección anual está entre el 70% y el 90%. Empezar a monitorear con mayor frecuencia.
- **Rojo:** la proyección supera el 90% del límite. Consultar con el contador para evaluar la recategorización.

■ *El porcentaje_usado que aparece en la pantalla refleja solo lo YA facturado. El semáforo se calcula sobre la proyección completa al 31/12. Son dos métricas distintas: podés estar en 48% real pero el semáforo estar en rojo si la tendencia proyecta superar el límite.*

7. Guía de troubleshooting

Síntoma	Causa probable	Solución
El clasificador devuelve 'Otros' con frecuencia para el 100% de los gastos	El modelo base no está cargado en memoria	Correr: docker compose exec api python seed_modelo_base Luego: docker compose restart api
POST /alertas/ejecutar-auditoria devuelve 500	Valor de num TipoAlerta no existe en PostgreSQL	Verificar que las migraciones 0004 y 0005 se aplicaron: docker compose exec api alembic current
El semáforo Monotributo muestra si el categoriao tiene categoría asignada o la tabla está vacía	El categoriao no tiene categoría asignada o la tabla está vacía	1. Asignar categoría: PATCH /monotributo/categoria 2. Si la tabla está vacía: python seed_categorias_monotributo
Importación CSV falla: 'No se pudo detectar la estructura'	El archivo no tiene encabezados o el formato es incorrecto	Verificar que la primera fila tenga nombres de columna. Verificar que el formato sea correcto
Login devuelve 401 aunque el password es correcto	El token es inválido o los tokens de sesión y refresh son inválidos	Crear sesión y refresh token para obtener un token nuevo
docker compose up falla: puerto en uso	Algo proceso usa el puerto 3000, 8000 o 5432	Verificar: lsof -i :8000 grep LISTEN O cambiar los puertos en docker-compose.yml
Las proyecciones no se generan (en el periodo)	El periodo no tiene ingresos históricos	Normal: el sistema usa media móvil automáticamente. Agregar datos históricos
El PDF descargado está vacío o mal renderizado (tablas muy anchas o datos nulos)	El periodo no tiene datos	Verificar logs: docker compose logs api grep pdf Asegurar que el usuario tiene datos en el período solicitado

7.1 Comandos de diagnóstico rápido

```
# Estado de los contenedores docker compose ps # Logs de la API en tiempo real docker
compose logs -f api # Estado de las migraciones docker compose exec api alembic current #
Aplicar migraciones pendientes docker compose exec api alembic upgrade head # Correr tests
dentro del contenedor docker compose exec api python -m pytest tests/ -q # Revisar el
estado del modelo ML curl http://localhost:8000/ml/estado -H 'Authorization: Bearer TOKEN'
# Salud general de la API curl http://localhost:8000/health
```

8. Preguntas frecuentes (FAQ)

P: ¿Por qué el clasificador corre local y no usa Groq/ChatGPT?

R: Por soberanía de datos: las descripciones de tus gastos contienen información financiera sensible. Si las enviáramos a un servicio externo, terceros tendrían acceso a tu historial de consumo. El clasificador local (SVM + TF-IDF) alcanza una precisión del ~76% sobre 12 categorías, más que suficiente para la tarea, sin sacrificar privacidad.

P: ¿Qué pasa si no tengo clave de Groq?

R: El sistema funciona perfectamente sin GROQ_API_KEY. Los endpoints de /resumen/financiero y /recomendaciones/ tienen fallbacks locales determinísticos que generan texto legible a partir de los datos. Solo perderás el tono más natural del LLM.

P: ¿Puedo importar extractos de cualquier banco?

R: El sistema reconoce los formatos más comunes de bancos argentinos: Galicia (con ;), Santander, Brubank, BBVA, Macro, Nación, Mercado Pago, entre otros. Si el tuyo no es detectado automáticamente, podés editar los encabezados del CSV para usar nombres reconocidos (fecha, concepto o descripcion, importe o monto).

P: ¿Cómo sé si el semáforo es confiable?

R: El semáforo es tan confiable como las proyecciones, que a su vez dependen de la regularidad de tus ingresos históricos. Con al menos 10 ingresos registrados, Prophet calcula la tendencia y la estacionalidad. Con menos datos usa media móvil. En ambos casos es una ESTIMACIÓN: siempre consultá con tu contador ante cambios significativos en tu nivel de facturación.

P: ¿Qué categoría de Monotributo debo elegir?

R: La categoría correcta depende de tu facturación de los últimos 12 meses y de tu actividad (servicios o comercio). El sistema calcula el acumulado móvil de 12 meses en el endpoint /monotributo/facturacion-12-meses. Con ese dato, consultá la tabla de la sección 4.8 de este manual o la página de AFIP.

P: ¿El reporte PDF reemplaza la información para el contador?

R: No. El PDF incluye un disclaimer explícito: 'No reemplaza el asesoramiento de un contador matriculado.' Es una herramienta de gestión y seguimiento, no un documento contable oficial.

P: ¿Puedo usar el sistema con múltiples usuarios?

R: Sí. El sistema está diseñado para ser multi-usuario desde su arquitectura: cada usuario tiene sus propios datos y modelos ML, completamente aislados. El usuario_id siempre se extrae del JWT, nunca del body de la solicitud.

P: ¿Qué pasa si borro el contenedor de PostgreSQL?

R: Los datos están en un volumen Docker (postgres_data). Si hacés 'docker compose down' sin -v, los datos persisten. Con 'docker compose down -v' se borran. Para hacer backup: docker compose exec db pg_dump -U marcos tfg_freelancers > backup.sql